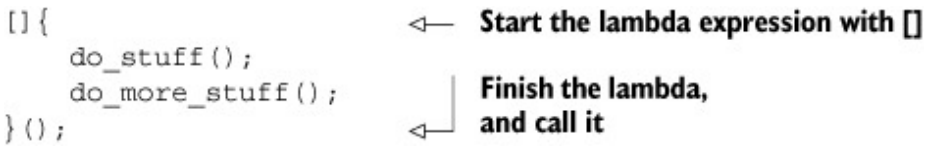
accessible variables rather than capturing the necessary state in the member variables of a class with a function call operator.

At its simplest, a *lambda expression* defines a self-contained function that takes no parameters and relies only on global variables and functions. It doesn't even have to return a value. Such a lambda expression is a series of statements enclosed in braces, prefixed with square brackets (the *lambda introducer*):



In this example, the lambda expression is called by following it with parentheses, but this is unusual. For one thing, if you're going to call it directly, you could usually do away with the lambda and write the statements directly in the source. It's more common to pass it as a parameter to a function template that takes a callable object as one of its parameters, in which case it likely needs to take parameters or return a value or both. If you need to take parameters, you can do this by following the lambda introducer with a parameter list just like for a normal function. For example, the following code writes all the elements of the vector to `std::cout` separated by newlines:

```
std::vector<int> data=make_data();
std::for_each(data.begin(),data.end(),[](int i)
{std::cout<<i<<"\n";});
```

Return values are almost as easy. If your lambda function body consists of a single `return` statement, the return type of the lambda is the type of the expression being returned. For example, you might use a simple lambda like this to wait for a flag to be set with a `std::condition_variable` (see section 4.1.1) as in the following listing.

Listing A.4. A simple lambda with a deduced return type